

CIA-2 Practical Examination

SQL and NoSQL Foundation

Name: ARPITHA RAICHEL JACOB

Reg Number: 2462607

Class: 5BTELCS

Practical Set 1 — Employee-Department Database

Attempted Tasks: Task 1, Task 2, Task 3, Task 5, Task 6, Task 7, Task 8, Task 9, Task 11, Task 13

Database & Table Setup

```
mysql> CREATE DATABASE company_db;
Query OK, 1 row affected (0.08 sec)

mysql> USE company_db;
Database changed

mysql> CREATE TABLE departments (
  -> dept_id INT PRIMARY KEY,
  -> dept_name VARCHAR(50),
  -> location VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.12 sec)

mysql> CREATE TABLE employees (
  -> emp_id INT PRIMARY KEY,
  -> emp_name VARCHAR(50),
  -> dept_id INT,
  -> salary DECIMAL(10,2),
  -> city VARCHAR(50),
  -> FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
  -> );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO departments VALUES
  -> (1, 'HR', 'Bangalore'),
  -> (2, 'IT', 'Chennai'),
  -> (3, 'Finance', 'Mumbai'),
  -> (4, 'Marketing', 'Delhi');
Query OK, 4 rows affected (0.02 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO employees VALUES
  -> (101, 'Arun Kumar', 1, 45000, 'Bangalore'),
  -> (102, 'Priya Singh', 2, 62000, 'Chennai'),
  -> (103, 'Ravi Shankar', 2, 58000, 'Hyderabad'),
  -> (104, 'Sneha Reddy', 3, 71000, 'Mumbai'),
  -> (105, 'Karthik Iyer', 2, 49000, 'Chennai'),
  -> (106, 'Divya Menon', 1, 39000, 'Bangalore'),
  -> (107, 'Vikram Rao', 3, 55000, 'Pune'),
  -> (108, 'Anjali Nair', 4, 47000, 'Delhi');
Query OK, 8 rows affected (0.03 sec)
Records: 8 Duplicates: 0 Warnings: 0
```

Creates the company_db database with two related tables, departments and employees, linked by a foreign key on dept_id. Sample rows are inserted into both tables to build the working dataset used by every task below.

Task 1: Employee-Department Database — Unions and Other Multiset Consolidations

```
mysql> SELECT city AS place FROM employees
-> UNION
-> SELECT location AS place FROM departments;
+-----+
| place |
+-----+
| Bangalore |
| Chennai |
| Hyderabad |
| Mumbai |
| Pune |
| Delhi |
+-----+
6 rows in set (0.01 sec)

mysql> SELECT city AS place FROM employees
-> UNION ALL
-> SELECT location AS place FROM departments;
+-----+
| place |
+-----+
| Bangalore |
| Chennai |
| Hyderabad |
| Mumbai |
| Chennai |
| Bangalore |
| Pune |
| Delhi |
| Bangalore |
| Chennai |
| Mumbai |
| Delhi |
+-----+
12 rows in set (0.01 sec)
```

UNION merges the employee cities and department locations into a single column and automatically removes duplicate values, returning 6 distinct places. UNION ALL performs the same merge but keeps every duplicate, returning all 12 combined rows from both tables.

Task 2: Employee-Department Database — Grouping Your Data

```
mysql> SELECT dept_id, SUM(salary) AS total_salary
-> FROM employees
-> GROUP BY dept_id;
+-----+-----+
| dept_id | total_salary |
+-----+-----+
|      1 |    84000.00 |
|      2 |   169000.00 |
|      3 |   126000.00 |
|      4 |    47000.00 |
+-----+-----+
4 rows in set (0.01 sec)
```

```
mysql> SELECT dept_id, COUNT(*) AS emp_count
-> FROM employees
-> GROUP BY dept_id;
+-----+-----+
| dept_id | emp_count |
+-----+-----+
|      1 |          2 |
|      2 |          3 |
|      3 |          2 |
|      4 |          1 |
+-----+-----+
4 rows in set (0.00 sec)
```

GROUP BY dept_id collapses the employees table into one row per department. SUM(salary) totals the salary bill for each department, while COUNT(*) counts how many employees belong to it.

Task 3: Employee-Department Database — The HAVING Clause

```
mysql> SELECT dept_id, COUNT(*) AS emp_count
-> FROM employees
-> GROUP BY dept_id
-> HAVING COUNT(*) > 3;
Empty set (0.00 sec)

mysql> SELECT dept_id, AVG(salary) AS avg_salary
-> FROM employees
-> GROUP BY dept_id
-> HAVING AVG(salary) > 50000;
+-----+-----+
| dept_id | avg_salary |
+-----+-----+
|      2 | 56333.3333 |
|      3 | 63000.0000 |
+-----+-----+
2 rows in set (0.01 sec)
```

HAVING filters groups after aggregation, unlike WHERE which filters individual rows beforehand. No department currently has more than 3 employees, so the first query correctly returns an empty set; the second isolates IT and Finance as the departments whose average salary exceeds 50000.

Task 5: Employee-Department Database — Table Joins

```
mysql> SELECT e.emp_name, d.dept_name, e.salary
-> FROM employees e
-> INNER JOIN departments d ON e.dept_id = d.dept_id;
+-----+-----+-----+
| emp_name | dept_name | salary |
+-----+-----+-----+
| Arun Kumar | HR | 45000.00 |
| Priya Singh | IT | 62000.00 |
| Ravi Shankar | IT | 58000.00 |
| Sneha Reddy | Finance | 71000.00 |
| Karthik Iyer | IT | 49000.00 |
| Divya Menon | HR | 39000.00 |
| Vikram Rao | Finance | 55000.00 |
| Anjali Nair | Marketing | 47000.00 |
+-----+-----+-----+
8 rows in set (0.01 sec)
```

```
mysql> SELECT d.dept_name, e.emp_name
-> FROM departments d
-> LEFT JOIN employees e ON d.dept_id = e.dept_id;
+-----+-----+
| dept_name | emp_name |
+-----+-----+
| HR | Arun Kumar |
| HR | Divya Menon |
| IT | Priya Singh |
| IT | Ravi Shankar |
| IT | Karthik Iyer |
| Finance | Sneha Reddy |
| Finance | Vikram Rao |
| Marketing | Anjali Nair |
+-----+-----+
8 rows in set (0.01 sec)
```

The INNER JOIN matches employees to departments on dept_id, returning only rows with a match on both sides. The LEFT JOIN keeps every department row regardless of whether it has employees — in this dataset every department already has staff, so the result looks the same, but a department with zero employees would still appear here with a NULL emp_name instead of being dropped.

Task 6: Employee-Department Database — Complex Multi Table Joins

```
mysql> CREATE TABLE projects (  
-> project_id INT PRIMARY KEY,  
-> project_name VARCHAR(50),  
-> emp_id INT,  
-> FOREIGN KEY (emp_id) REFERENCES employees(emp_id)  
-> );  
Query OK, 0 rows affected (0.09 sec)  
  
mysql> INSERT INTO projects VALUES  
-> (1, 'Payroll System', 101),  
-> (2, 'Website Revamp', 102),  
-> (3, 'Cloud Migration', 103),  
-> (4, 'Budget Analysis', 104),  
-> (5, 'Mobile App', 105);  
Query OK, 5 rows affected (0.02 sec)  
Records: 5  Duplicates: 0  Warnings: 0  
  
mysql> SELECT e.emp_name, d.dept_name, p.project_name  
-> FROM employees e  
-> JOIN departments d ON e.dept_id = d.dept_id  
-> JOIN projects p ON e.emp_id = p.emp_id;  
+-----+-----+-----+  
| emp_name | dept_name | project_name |  
+-----+-----+-----+  
| Arun Kumar | HR | Payroll System |  
| Priya Singh | IT | Website Revamp |  
| Ravi Shankar | IT | Cloud Migration |  
| Sneha Reddy | Finance | Budget Analysis |  
| Karthik Iyer | IT | Mobile App |  
+-----+-----+-----+  
5 rows in set (0.01 sec)
```

A third table, projects, is added and linked to employees through emp_id. Joining all three tables in one query pulls together an employee's name, their department, and the project they are assigned to — only employees who actually have a project row appear in the result.

Task 7: Employee-Department Database — Filtering Using Subqueries

```
mysql> SELECT emp_name, salary
-> FROM employees
-> WHERE salary > (SELECT AVG(salary) FROM employees);
```

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Priya Singh | 62000.00 |
| Ravi Shankar | 58000.00 |
| Sneha Reddy | 71000.00 |
| Vikram Rao | 55000.00 |
+-----+-----+
4 rows in set (0.01 sec)
```

```
mysql> SELECT emp_name, dept_id
-> FROM employees
-> WHERE dept_id = (
->     SELECT dept_id FROM employees
->     GROUP BY dept_id
->     ORDER BY COUNT(*) DESC
->     LIMIT 1
-> );
```

```
+-----+-----+
| emp_name | dept_id |
+-----+-----+
| Priya Singh | 2 |
| Ravi Shankar | 2 |
| Karthik Iyer | 2 |
+-----+-----+
3 rows in set (0.01 sec)
```

The inner subquery in the first query computes the company-wide average salary (53250), and the outer query filters employees earning above it. The second query's subquery finds the dept_id with the most employees — IT, with 3 — and the outer query returns just that department's staff.

Task 8: Employee-Department Database — The EXISTS Clause

```
mysql> SELECT d.dept_name
-> FROM departments d
-> WHERE EXISTS (
->     SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id
-> );
```

```
+-----+
| dept_name |
+-----+
| HR        |
| IT        |
| Finance   |
| Marketing |
+-----+
4 rows in set (0.01 sec)
```

```
mysql> SELECT d.dept_name
-> FROM departments d
-> WHERE NOT EXISTS (
->     SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id
-> );
Empty set (0.00 sec)
```

EXISTS checks, for each department row, whether the correlated subquery finds any matching employee, without caring how many. Since all four departments currently have at least one employee, the EXISTS query returns all of them and the NOT EXISTS query correctly returns nothing.

Task 9: Employee-Department Database — Derived Tables

```
mysql> SELECT dept_avg.dept_id, dept_avg.avg_salary
-> FROM (
->     SELECT dept_id, AVG(salary) AS avg_salary
->     FROM employees
->     GROUP BY dept_id
-> ) AS dept_avg
-> WHERE dept_avg.avg_salary > 45000;
+-----+-----+
| dept_id | avg_salary |
+-----+-----+
|      2 | 56333.3333 |
|      3 | 63000.0000 |
|      4 | 47000.0000 |
+-----+-----+
3 rows in set (0.01 sec)
```

The inner query is treated as a temporary "derived table" named dept_avg, computing the average salary per department first. The outer query then filters that already-aggregated result for averages above 45000, which HR (42000) fails to meet.

Task 11: Employee-Department Database — Transactions, Initiating a Transaction

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE employees SET salary = salary + 2000 WHERE emp_id = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE employees SET salary = salary + 3000 WHERE emp_id = 103;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO employees VALUES (109, 'Meena Pillai', 4, 52000, 'Delhi');
Query OK, 1 row affected (0.01 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

START TRANSACTION opens a block in which multiple statements are grouped as one all-or-nothing unit. Two salary updates and a new employee insert are made inside the transaction, and COMMIT then makes all three changes permanent together — had something gone wrong before the COMMIT, a ROLLBACK would have undone all three at once instead of leaving the data half-changed.

Task 13: Employee-Department Database — Creating Tables, Creating and Using Views

```
mysql> CREATE VIEW high_salary_employees AS
-> SELECT emp_id, emp_name, dept_id, salary
-> FROM employees
-> WHERE salary > 50000;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> SELECT * FROM high_salary_employees;
+-----+-----+-----+-----+
| emp_id | emp_name      | dept_id | salary  |
+-----+-----+-----+-----+
| 102    | Priya Singh   | 2       | 62000.00 |
| 103    | Ravi Shankar  | 2       | 61000.00 |
| 104    | Sneha Reddy   | 3       | 71000.00 |
| 107    | Vikram Rao    | 3       | 55000.00 |
+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

CREATE VIEW saves this query as a reusable virtual table, `high_salary_employees`, without physically duplicating data. Selecting from the view returns the current employees earning above 50000 based on the live table — including Ravi Shankar's updated salary of 61000 from Task 11 — exactly as if the underlying SELECT had been run directly.

Anitha Racheel Jacob
2462607
SBTELS

Theory
Data Engineering Foundation
(SQL & NOSQL)

Attempted question [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

1. UNION
- 2) SELECT department, SUM(salary) AS total-salary
FROM employees
GROUP BY DEPARTMENT;
- 3) HAVING
- 4) AS
- 5) INNER JOIN
- 6) SELECT student . student_name,
courses . course_name,
enrollment . marks.

FROM students
INNER JOIN enrollments
ON students . student_id = enrollment
. student_id

INNER JOIN courses
ON enrollment . course_id = courses . course_id

Aspitha Rachel Jacobo

5BTELCS

(8Q1E, NOSQ1)

Attempted question [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

2) SELECT department, SUM (salary) AS total-salary
FROM employees
GROUP BY DEPARTMENT;

4) AS

b) `SELECT student . student_name ,
courses . course_name ,
enrollment . marks .`

INNER JOIN emplacements

ON Students. Student_id = Enrollment
INNER JOIN Courses Student_id

INNER JOIN courses ON enrollment, course_id = courses_course_id

7 Subquery

8. Exists

9. Derived

10. UPDATE employees

SET salary = 5500

WHERE id = 5;

